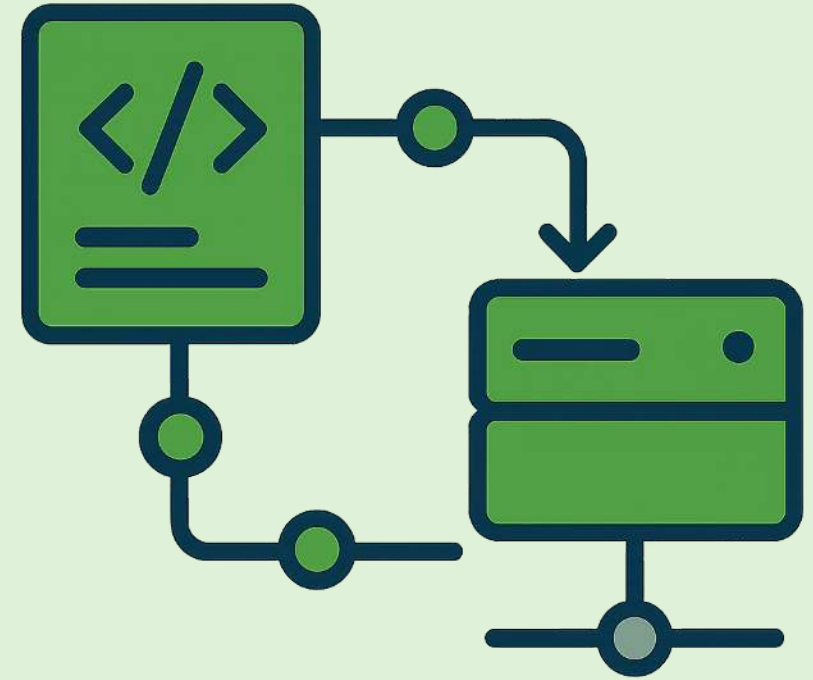


Microservices – Zusatz UE

CI/CD mit GitHub Actions

Automatisierung von Build, Test und Deployment in modernen
Softwareprojekten



Agenda

01

Rückblick

Was bisher in Docker & Kubernetes erarbeitet wurde

02

CI/CD Grundlagen

Konzepte, Problemstellung und Pipeline-Überblick

03

GitHub Actions

Einführung, Workflow-Struktur und Grundbegriffe

04

Praxis

Build, Test, Docker-Build und Deployment automatisieren

05

Verbindung

Docker & Microservices in der Pipeline

Lernziele

Nach dieser Einheit kannst du:

→ **CI/CD erklären**

Du verstehst Continuous Integration und Deployment als Konzept

→ **Workflows lesen**

GitHub Actions YAML-Workflows verstehen und interpretieren

→ **Automatisch bauen & testen**

Code per Pipeline kompilieren und Testsuites ausführen

→ **Docker-Images per Pipeline erstellen**

Dockerfile automatisch bauen und in Registry pushen

→ **Deployments automatisieren**

Einfache Rollouts ohne manuelle Eingriffe durchführen

Rückblick: Was bisher erarbeitet wurde

UE2 – Docker & Dockerfiles

Container-Images erstellen und verwalten

UE3 – Docker Compose

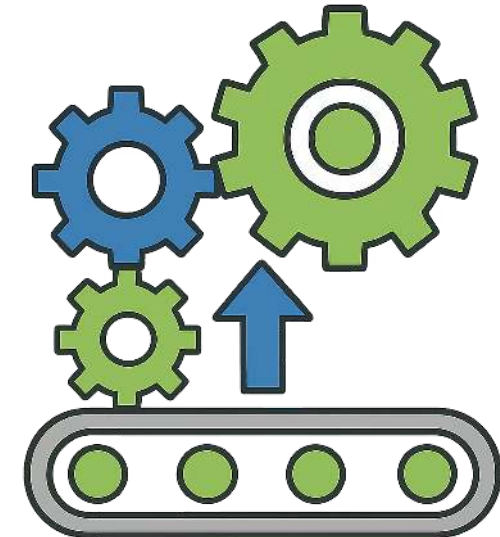
Mehrere Services gemeinsam orchestrieren

UE4 – Kubernetes & Deployment

Skalierung und Verwaltung von Containern im Cluster

? Frage zur Reflexion

Wie automatisieren wir diese Schritte – von Dockerfile über Compose bis zum Kubernetes-Deployment – ohne jeden Schritt manuell auszuführen?





Das Problem: Manuelles Deployment



Fehleranfällig

Manuelle Schritte bedeuten menschliche Fehler – vergessene Befehle, falsche Versionen



Langsam

Jedes Deployment kostet Zeit und blockiert das Entwicklungsteam unnötig



Nicht reproduzierbar

Unterschiedliche Ergebnisse je nach Person, Maschine oder Zeitpunkt

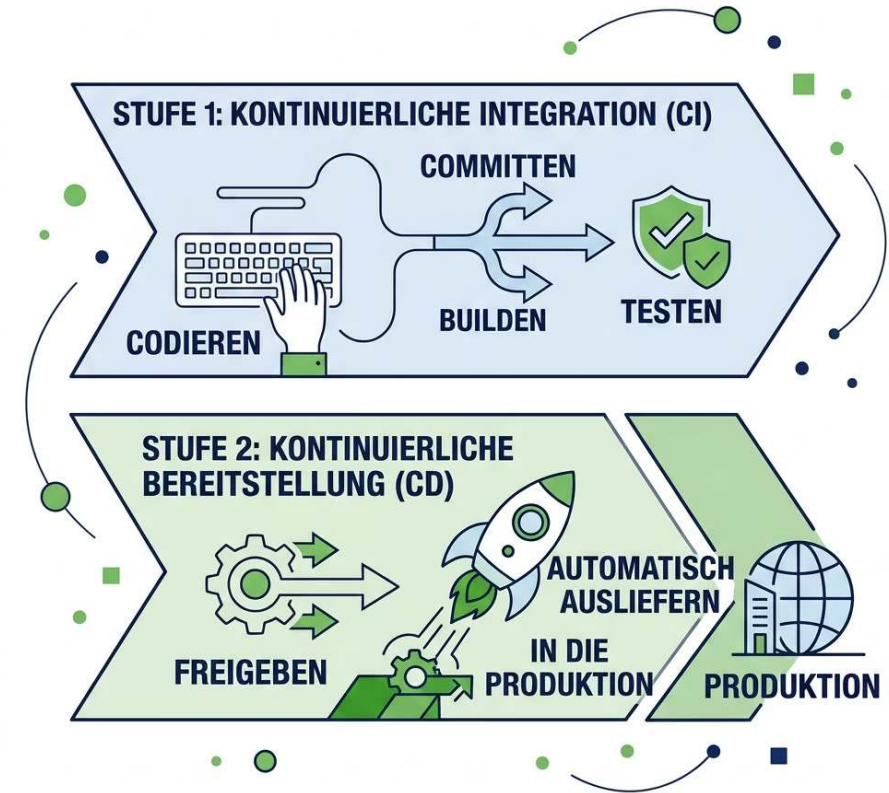
Einführung CI/CD

CI – Continuous Integration

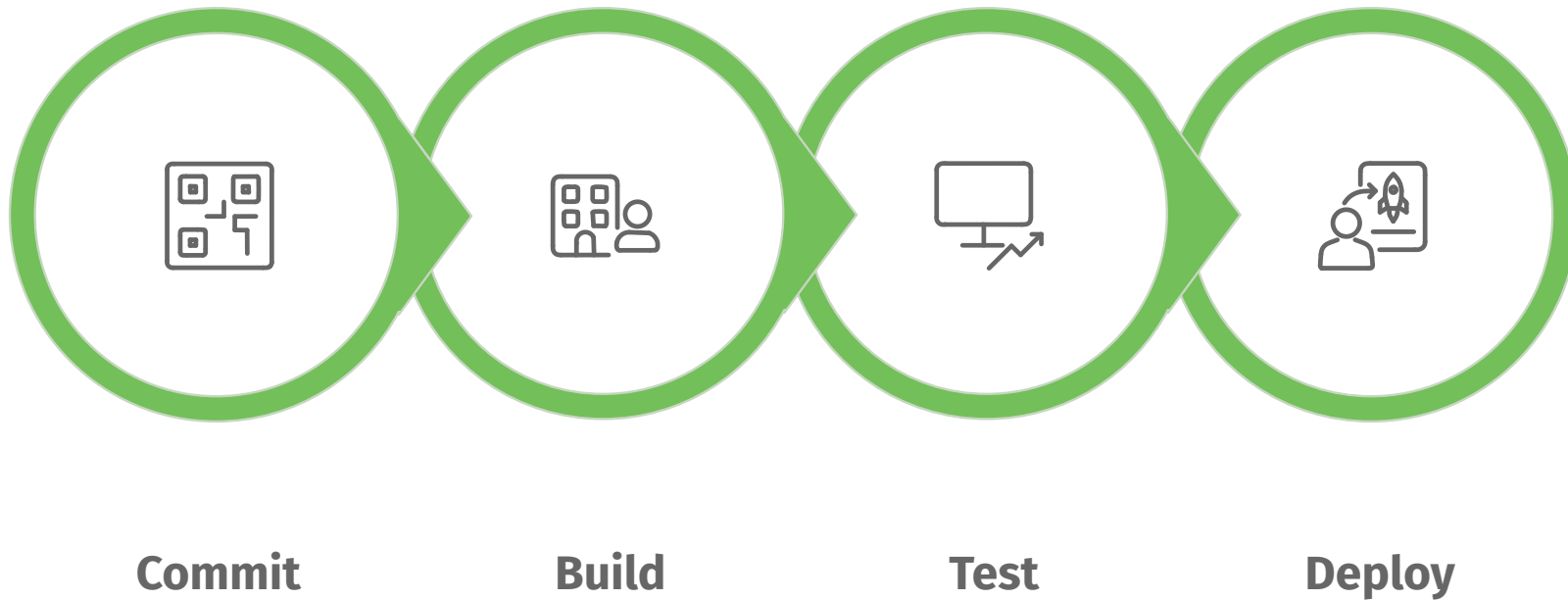
Code wird bei jedem Commit **automatisch getestet und gebaut**. Fehler werden früh erkannt, bevor sie sich anhäufen.

CD – Continuous Deployment

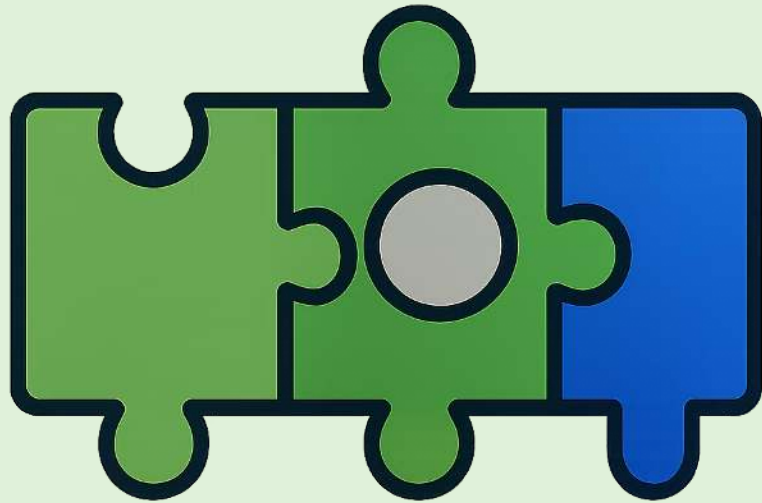
Gebauter und getesteter Code wird **automatisch in die Zielumgebung ausgerollt** – ohne manuellen Eingriff.



Typische Pipeline-Schritte



Jeder Commit durchläuft diese Schritte automatisch – von der ersten Codezeile bis zum fertigen Deployment.



Verbindung zu euren Projekten

Was ihr bereits erstellt habt:

Dockerfile (UE2)

Definiert, wie euer Container-Image gebaut wird

Docker Compose Setup (UE3)

Orchestriert mehrere Services zusammen

- ✓ **Jetzt:** Diese Schritte werden mit GitHub Actions vollständig automatisiert – kein manuelles `docker build` oder `docker compose up` mehr.



GitHub Actions

GitHub Actions ist das integrierte CI/CD-Tool von GitHub. Kein separates System nötig – die Pipeline lebt direkt im Repository.

Basiert auf YAML

Workflows werden als YAML-Dateien im Repo versioniert

Event-gesteuert

Läuft bei Events wie `push` oder `pull_request`

Workflow-Struktur

```
name: CI
```

```
on: [push]
```

```
jobs:
```

```
  build:
```

```
    runs-on: ubuntu-latest
```

name

Bezeichnung des Workflows – erscheint in der GitHub UI

on

Definiert den Trigger – hier: bei jedem Push

jobs

Enthält einen oder mehrere Jobs, die ausgeführt werden

runs-on

Die Runner-Umgebung – typischerweise `ubuntu-latest`

Grundbegriffe in GitHub Actions



Workflow

Die gesamte Automatisierungsdefinition in einer YAML-Datei unter `.github/workflows/`



Job

Eine Gruppe von Steps, die auf einem Runner ausgeführt werden. Jobs können parallel oder sequenziell laufen.



Step

Ein einzelner Befehl oder eine Action innerhalb eines Jobs – der kleinste ausführbare Baustein



Runner

Die virtuelle Maschine, auf der der Job ausgeführt wird (z. B. `ubuntu-latest`)

Erste Pipeline – Hello CI

steps:

- uses: actions/checkout@v4
- run: **echo** "Hello CI"

Was passiert hier?

1

actions/checkout@v4

Lädt den Repository-Code auf den Runner – erster Step in fast jedem Workflow

2

run: echo "Hello CI"

Führt einen Shell-Befehl aus – hier eine einfache Ausgabe als Proof of Concept

Ersten Workflow lesen und auslösen

Ziel

Einen minimalen GitHub-Actions-Workflow verstehen.

 Arbeitsordner: `uebung-16-erster-workflow`

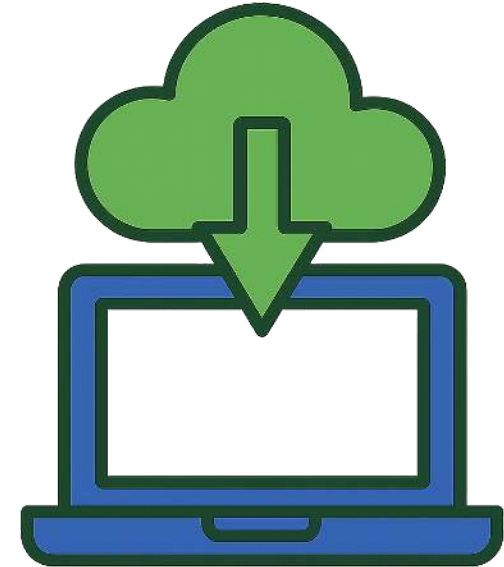
Aufgaben

1. Öffne `.github/workflows/ci.yml`
2. Identifiziere Trigger, Job und Steps
3. Erkläre, wann der Workflow läuft
4. Benenne, welche Ausgabe der Workflow erzeugt

Code auschecken

- uses: actions/checkout@v4

- ❗ Dieser Step ist in nahezu jedem Workflow der erste – ohne ihn hat der Runner keinen Zugriff auf den Code im Repository.

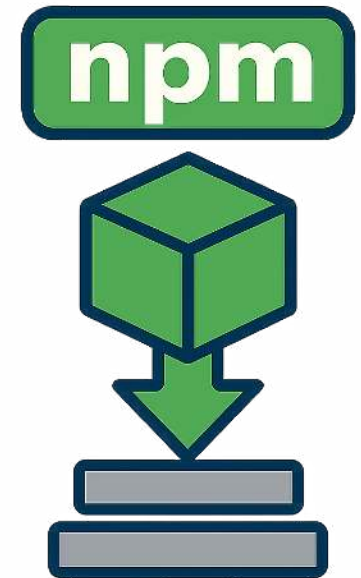


Build-Schritt: Node-Anwendung

Abhängigkeiten installieren

- run: npm install

Dieser Step lädt alle im `package.json` definierten Abhängigkeiten herunter und macht die Anwendung build-fähig.



Test-Schritt



AUTOMATED SOFTWARE TESTING

Tests automatisch ausführen

```
- run: npm test
```

Führt die im Projekt definierten Tests aus. Schlägt ein Test fehl, stoppt die Pipeline – kein fehlerhafter Code wird weitergeleitet.

- ✅ Früh scheitern ist gut: Fehler werden erkannt, bevor Code deployed wird.

Test-Workflow nachvollziehen

Ziel

Verstehen, wie eine Pipeline eine kleine Anwendung installiert und testet.

 Arbeitsordner: `uebung-17-test-workflow`

Aufgaben

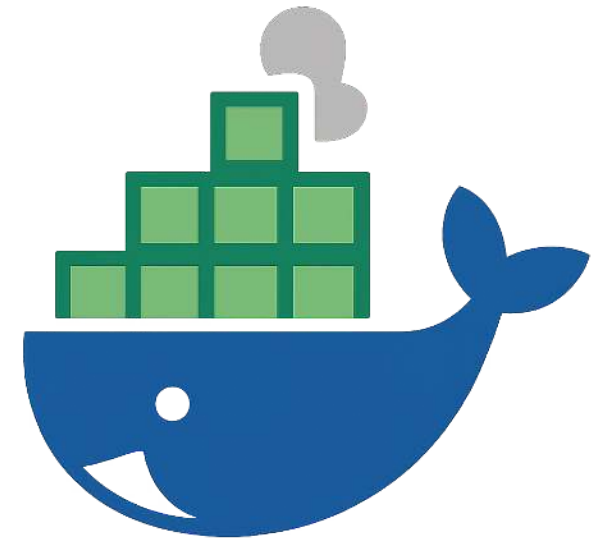
1. Öffne die kleine Python-Anwendung und den Workflow
2. Finde heraus, in welcher Reihenfolge `install` und `test` laufen
3. Benenne, welche Datei die Tests ausführt
4. Erkläre, was bei einem fehlschlagenden Test passieren würde

Docker Build in der Pipeline

Image automatisch bauen

```
- run: docker build -t myapp .
```

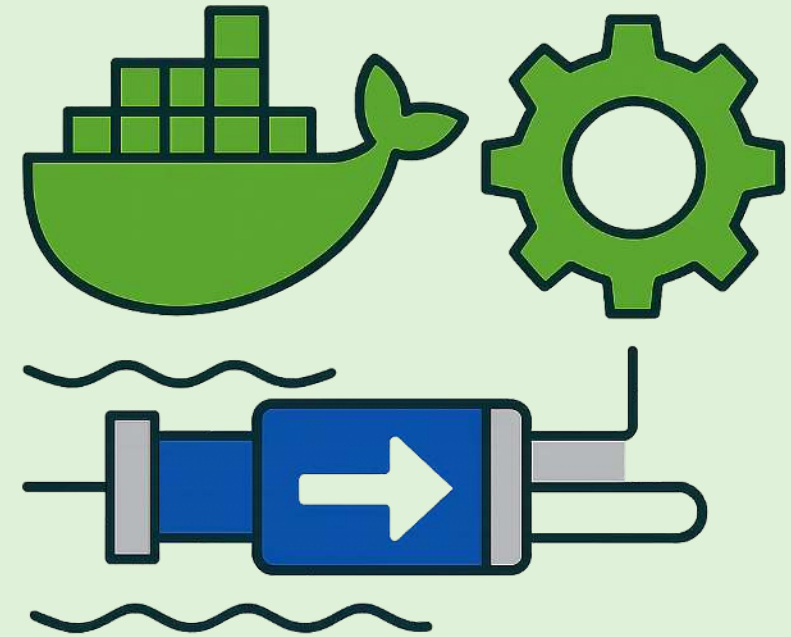
Der Runner führt `docker build` aus und erstellt ein Container-Image direkt aus dem Dockerfile im Repository – vollständig automatisiert.



Verbindung zu UE2

- ✓ Das Dockerfile aus UE2 wird jetzt automatisch gebaut – bei jedem Commit, ohne manuellen Befehl.

Was früher manuell mit `docker build` auf dem eigenen Rechner ausgeführt wurde, übernimmt jetzt die Pipeline – konsistent, reproduzierbar und auf einem definierten Runner.



Docker Build per Workflow verstehen

Ziel

Nachvollziehen, wie ein Workflow ein Docker Image baut.

 Arbeitsordner: `uebung-18-docker-build-workflow`

Aufgaben

1. Öffne das Dockerfile und den Workflow
2. Finde den Step, der das Image baut
3. Erkläre, welche Dateien dafür im Repository liegen müssen
4. Benenne, was der Runner dafür lokal bereitstellt

Docker Login für Registry-Zugriff



Authentifizierung gegenüber der Registry

- run: docker login

Bevor ein Image in eine Registry gepusht werden kann, muss sich der Runner authentifizieren. In der Praxis geschieht dies mit Secrets – nicht mit Klartext-Credentials im Workflow.

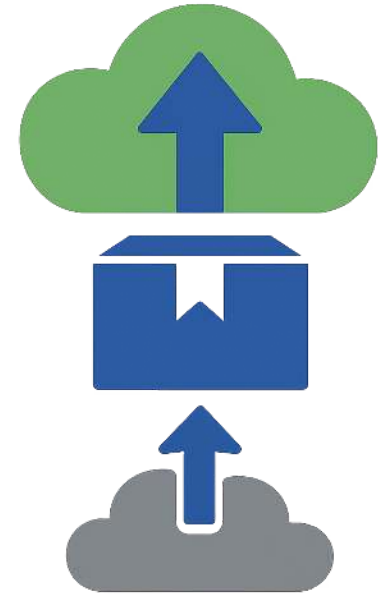
⚠️ Passwörter gehören niemals direkt in die YAML-Datei!

Docker Image pushen

Image in die Registry übertragen

```
- run: docker push myapp
```

Nach dem Build wird das Image in eine Container-Registry hochgeladen (z. B. Docker Hub oder GitHub Container Registry). Von dort kann es auf beliebigen Servern oder in Kubernetes deployments werden.



Multi-Job Pipeline mit Artifact lesen

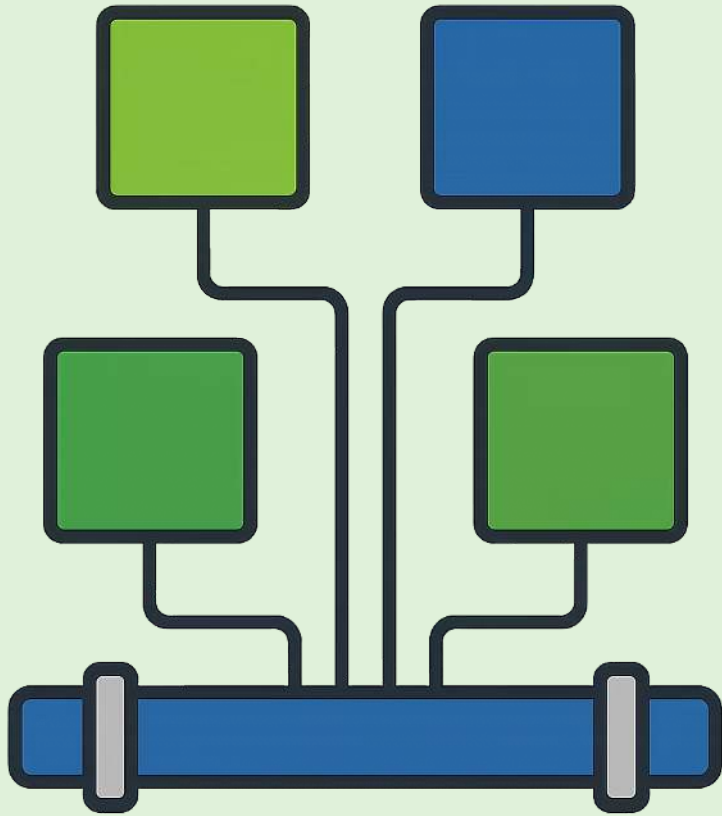
Ziel

Verstehen, wie Jobs getrennt laufen und Daten per Artifact weitergeben.

📁 Arbeitsordner: uebung-19-multi-job-artifact

Aufgaben

1. Öffne den Workflow im Ordner
2. Identifiziere den Build-Job und den nachgelagerten Test-Job
3. Benenne das erzeugte Artifact
4. Erkläre, warum `needs:` in diesem Workflow wichtig ist



Multi-Service Bezug

In einer Microservices-Architektur gibt es nicht ein Image – sondern viele.

Mehrere Images

Jeder Microservice hat sein eigenes Dockerfile und wird als eigenständiges Image gebaut

Mehrere Pipelines oder Jobs

Services können in parallelen Jobs oder separaten Workflows gebaut werden

Beispiel: Multi-Job Pipeline

jobs:

service-a:

runs-on: ubuntu-latest

steps:

- run: docker build -t service-a .

service-b:

runs-on: ubuntu-latest

steps:

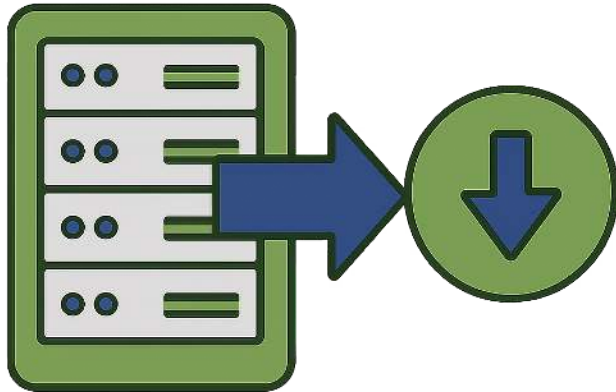
- run: docker build -t service-b .

Parallelität nutzen

Mehrere Jobs laufen gleichzeitig auf separaten Runnern. Jeder Service wird unabhängig gebaut – das spart Zeit und isoliert Fehler.

- ❗ Mit `needs:` lässt sich eine Abhängigkeit erzwingen – z. B. erst deployen, wenn beide Services gebaut sind.

Deployment-Schritt



Typisches Deployment via SSH

01

SSH auf Server

Verbindung zum Zielsystem über
gespeicherten SSH-Key (Secret)

02

`docker pull`

Neues Image aus der Registry laden

03

`docker compose up`

Services mit dem neuen Image starten

Secret und Deploy-Workflow einordnen

Ziel

Verstehen, wie Secrets und ein manueller Deploy-Schritt in GitHub Actions eingebunden werden.

 Arbeitsordner: uebung-20-secret-und-deploy

Aufgaben

1. Öffne den Workflow
2. Identifiziere den manuellen Trigger
3. Finde die Stelle, an der ein Secret verwendet wird
4. Erkläre, warum ein Deploy-Job oft nicht bei jedem Push laufen sollte

Typische Fehler in GitHub Actions

Falscher Workflow-Trigger

Der Workflow läuft nicht, weil `on: push` auf dem falschen Branch oder Event konfiguriert ist

Fehlende Secrets

Secrets wurden im Repository nicht hinterlegt – Authentifizierung schlägt fehl

Docker nicht verfügbar

Der Runner hat keinen Docker-Daemon oder die Action fehlt

Falsche Pfade

`docker build .` findet kein Dockerfile, weil der Pfad nicht zum Repository-Layout passt

Secrets in GitHub

Kein Klartext im Code

Passwörter, API-Keys und SSH-Keys dürfen **niemals** direkt in YAML-Dateien stehen – sie wären im Repository öffentlich sichtbar.

GitHub Secrets nutzen

Secrets werden in den Repository-Einstellungen hinterlegt und stehen dem Workflow als verschlüsselte Umgebungsvariablen zur Verfügung.

- ✔ Secrets erscheinen in Logs maskiert – auch bei versehentlichem `echo`.



Secret-Nutzung im Workflow

env:

```
PASSWORD: ${ secrets.PASSWORD }
```

Secrets werden über die `${ secrets.NAME }`-Syntax referenziert und als Umgebungsvariable in den Step injiziert.

Schritt 1: Secret anlegen

Im GitHub Repository unter *Settings* → *Secrets* → *Actions* einen neuen Eintrag erstellen

Schritt 2: Im Workflow referenzieren

Mit `${ secrets.MEIN_SECRET }` im YAML verwenden

Schritt 3: Sicher übergeben

Das Secret wird verschlüsselt übergeben und nie im Klartext geloggt

Diskussion

“

Was würdet ihr automatisieren?

Überlegt: Welche Schritte in euren bisherigen Projekten hätten von einer Pipeline profitiert?

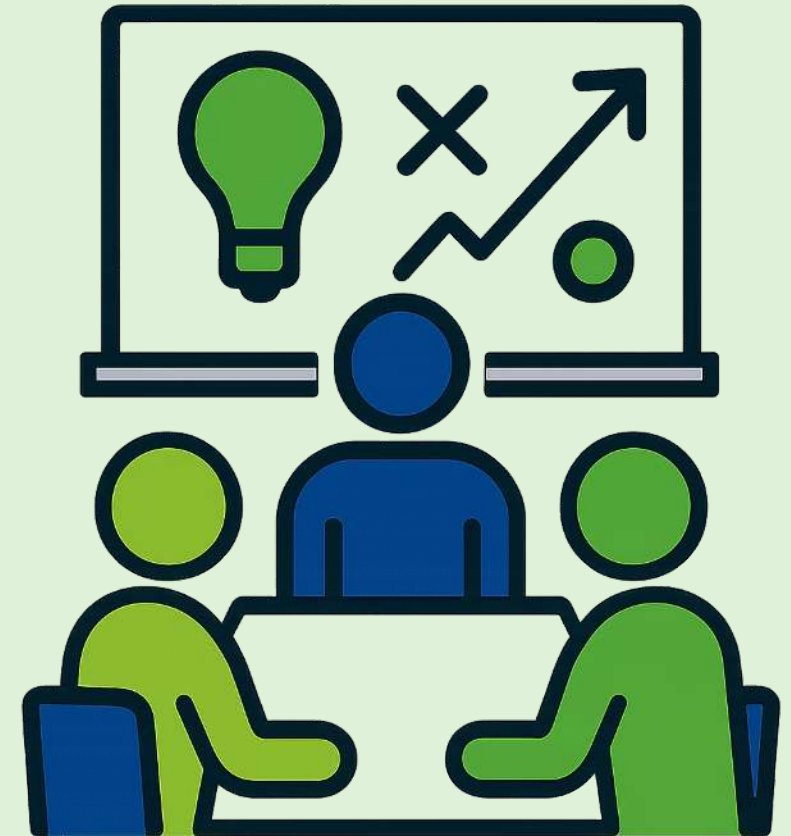
”

“

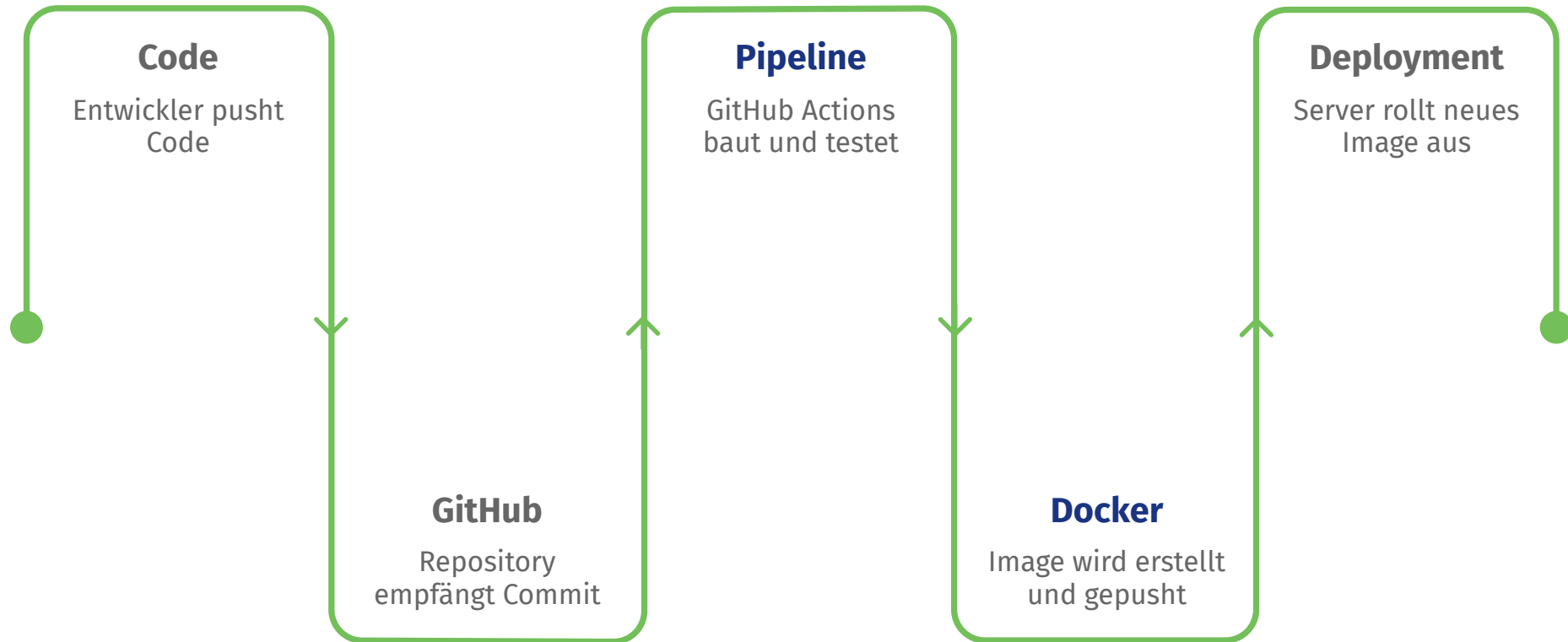
Wo seht ihr Risiken?

Vollautomatisches Deployment kann auch Probleme erzeugen – wann ist manuelles Eingreifen sinnvoll?

”



Das Gesamtbild



Diese Kette läuft vollautomatisch – vom ersten Commit bis zum laufenden Container auf dem Zielserver.

Komplette Pipeline entwerfen

Entwerft eine vollständige CI/CD-Pipeline für euren Microservice. Notiert eure Antworten als Stichpunkte im Chat.

1

Code Push

Welcher Trigger? Welcher Branch?

2

Tests

Welche Testbefehle? Welche Abhängigkeiten?

3

Docker Build

Welches Dockerfile? Welcher Image-Name?

4

Deployment

Wohin wird deployed? Welche Secrets werden benötigt?

Zusammenfassung

CI/CD automatisiert Prozesse

Von manuellem, fehleranfälligem Deployment zu reproduzierbaren, automatischen Abläufen

GitHub Actions als Tool

YAML-basierte Workflows direkt im Repository – event-gesteuert, einfach versionierbar

Verbindung zu Docker & Microservices

Dockerfiles, Images und Multi-Service-Setups werden nahtlos in die Pipeline integriert



Abschluss & Ausblick

Fragen?

Jetzt ist Zeit für offene Fragen zu den Übungen, den Konzepten oder der Anwendung auf eigene Projekte.

Weiterführende Themen

- **Matrix Builds**
Mehrere Umgebungen parallel testen
- **Reusable Workflows**
Workflows modular wiederverwenden
- **GitOps & ArgoCD**
Deployment-Automatisierung für Kubernetes